

TP Noté DevOps

Observabilité (Grafana) & CI/CD

Nicolas GOUY

Parcours : Docker Compose

Juin 2026

Dépôt GitHub : https://github.com/gonicolas12/tp_observ

1. Introduction

Ce TP a pour objectif de mettre en œuvre deux composantes essentielles d'une infrastructure DevOps moderne : la supervision active d'une application conteneurisée et l'automatisation du cycle de vie de son image Docker. Le premier livrable consiste en une stack de supervision complète associant Prometheus, node_exporter, cAdvisor, Grafana et Alertmanager, avec diffusion des alertes par e-mail (via MailHog) et via un webhook Discord. Le second livrable est un pipeline CI/CD GitHub Actions qui orchestre les tests unitaires, la construction de l'image Docker et son analyse de sécurité par Trivy.

Le parcours Docker Compose a été retenu car l'application TaskBoard — composée d'un frontend HTML/JS, d'un backend Node.js/Express et d'une base de données PostgreSQL — était déjà entièrement conteneurisée. Ce choix permet d'intégrer les services de supervision dans le même fichier Compose, en préservant l'isolation réseau et la simplicité de déploiement local.

Partie A — Stack de supervision

La chaîne de supervision s'articule autour de Prometheus comme moteur central de collecte et d'évaluation. Deux exporteurs alimentent Prometheus en métriques : node_exporter expose les métriques système de l'hôte (CPU, RAM, réseau, disque), tandis que cAdvisor expose les métriques propres à chaque conteneur Docker. Prometheus scrute ces deux sources à intervalles réguliers et évalue en parallèle les règles d'alerte définies dans alerts.yml. Lorsqu'une condition est satisfaite au-delà du délai configuré, Prometheus déclenche Alertmanager, qui se charge du routage vers les deux receivers configurés : un serveur SMTP simulé (MailHog) et un webhook Discord. Grafana est connecté à Prometheus comme datasource et permet la visualisation en temps réel via un dashboard provisionné automatiquement au démarrage de la stack.

A.1 — Cibles Prometheus (Status > Targets)

La page Status > Targets de Prometheus confirme que les quatre jobs de scrape sont opérationnels. Le job alertmanager (port 9093), le job cadvisor (port 8080), le job node (node-exporter, port 9100) et le job prometheus (port 9090 local) sont tous à l'état UP, ce qui valide la configuration des scrape_configs dans prometheus.yml et la bonne connectivité réseau entre les conteneurs Docker.

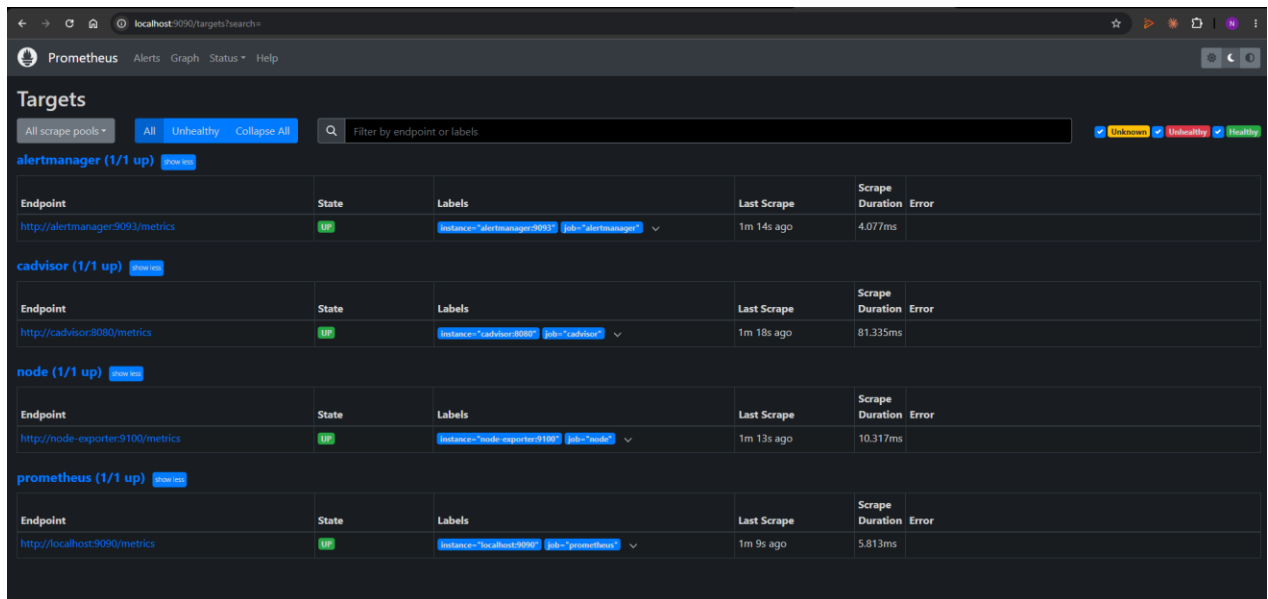


Figure 1 — Prometheus, Status > Targets : les quatre cibles (alertmanager, cadvisor, node, prometheus) sont UP

A.2 — Dashboard Grafana

La datasource Prometheus et le dashboard "TaskBoard - Vue d'ensemble" sont provisionnés automatiquement au démarrage grâce aux mécanismes de provisioning de Grafana (fichiers YAML montés dans /etc/grafana/provisioning/). Aucune configuration manuelle n'est nécessaire.

Le dashboard expose six panneaux complémentaires : un indicateur d'état node_exporter UP/DOWN, un graphe de l'utilisation CPU de l'hôte en pourcentage, une jauge de RAM disponible affichant 82,1 %, deux graphes cAdvisor montrant respectivement le CPU et la mémoire consommés par chaque conteneur (taskboard-backend, taskboard-db, taskboard-frontend), et un panneau de trafic réseau (octets reçus/émis sur eth0 et loopback).

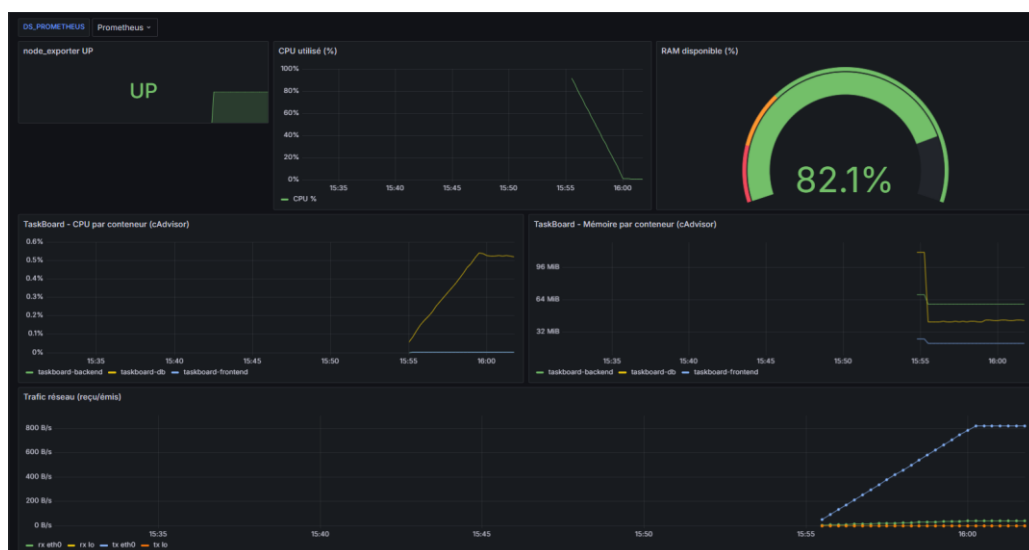


Figure 2 — Dashboard Grafana "TaskBoard - Vue d'ensemble" : état node_exporter, CPU %, jauge RAM, CPU/mémoire par conteneur (cAdvisor), trafic réseau

A.3 — Alerting e-mail et Discord

La règle d'alerte `NodeExporterDown` est définie dans `alerts.yml` avec l'expression `up{job="node"} == 0` maintenue pendant 1 minute, et le label `severity: critical`. Pour valider le circuit complet, le conteneur `node_exporter` a été arrêté volontairement (`docker compose stop node-exporter`). Après la période de grâce d'une minute, Prometheus fait passer l'alerte en état `FIRING` et la pousse vers Alertmanager.

Alertmanager v0.27 intègre nativement le support de `discord_configs` dans sa configuration de routes, ce qui permet de router la même alerte simultanément vers deux receivers distincts : le receiver `email` (pointant vers MailHog sur le port 1025) et le receiver `discord` (utilisant un webhook vers un salon dédié). MailHog joue le rôle de faux serveur SMTP — il accepte tous les e-mails sans les acheminer réellement, ce qui évite toute dépendance à un vrai compte SMTP lors du développement.

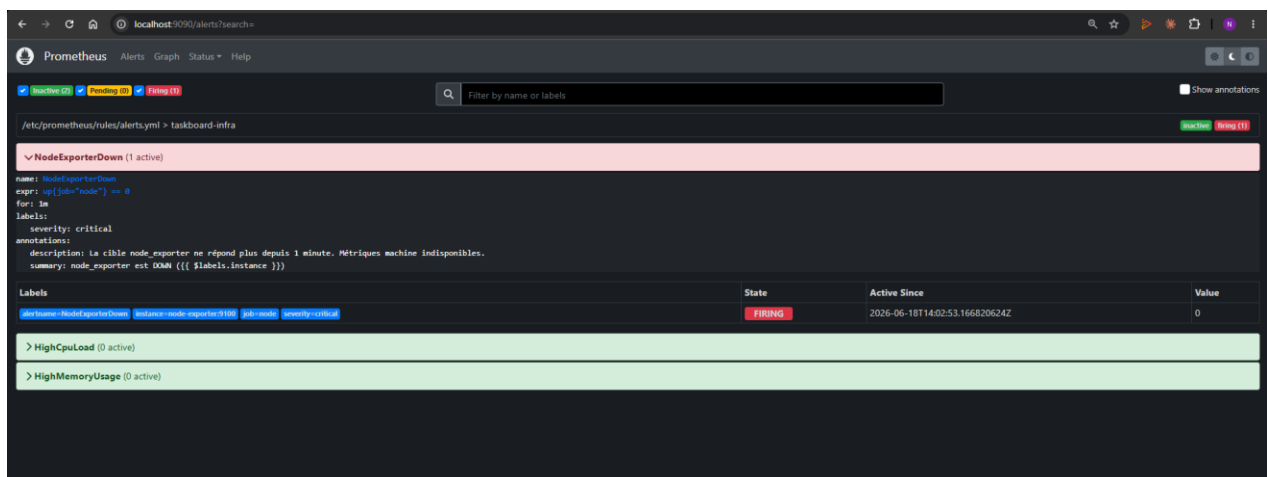


Figure 3 — Prometheus, page Alerts : alerte "NodeExporterDown" à l'état `FIRING` (active depuis le 18/06/2026 à 14:02:53)

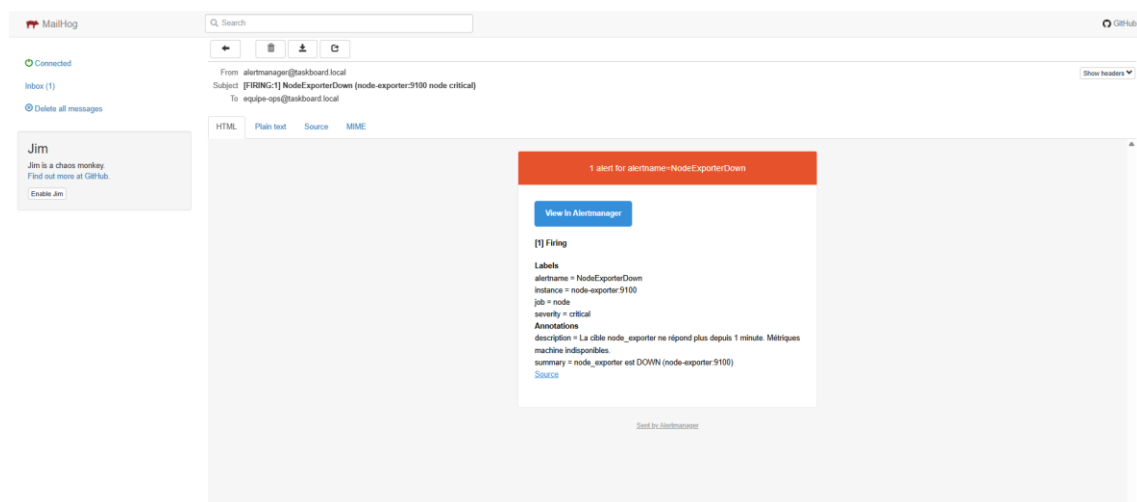


Figure 4 — MailHog : réception du mail d'alerte "[FIRING:1] NodeExporterDown (node-exporter:9100 node critical)" avec les labels et annotations complets

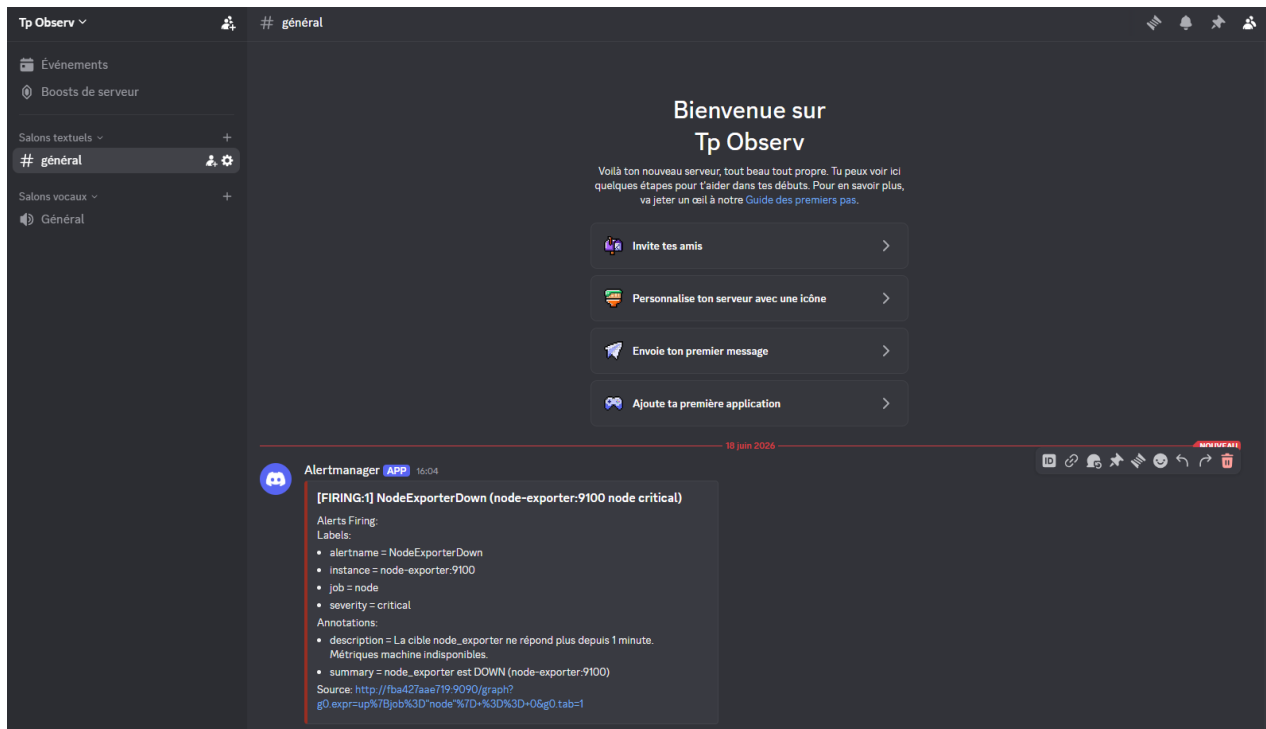


Figure 5 — Discord : message "[FIRING:1] NodeExporterDown (node-exporter:9100 node critical)" publié automatiquement dans le salon #général par l'application Alertmanager via webhook

Partie B — Pipeline CI/CD (GitHub Actions)

Le pipeline est défini dans `.github/workflows/ci.yml` et se déclenche automatiquement à chaque push sur la branche principale. Il est structuré en trois jobs séquentiels qui doivent tous réussir pour valider le push.

Le premier job, Tests unitaires, installe les dépendances Node.js et exécute la suite de tests unitaires du backend avec `npm test`. Il valide la logique applicative avant d'engager la construction de l'image.

Le deuxième job, Build & Scan Trivy, construit l'image Docker du backend avec `docker build`, puis lance Trivy avec les options `--severity HIGH,CRITICAL` et `--exit-code 1`. Ce paramètre est essentiel : il force le job à échouer (exit code 1) dès qu'une vulnérabilité de sévérité HIGH ou CRITICAL est détectée, bloquant ainsi la suite du pipeline.

Le troisième job, Deploy (bonus), simule une étape de déploiement et ne s'exécute que si les deux jobs précédents ont réussi.

B.1 — Pipeline rouge : détection de CVE par Trivy

Lors du premier run, Trivy a détecté plusieurs vulnérabilités HIGH dans l'image construite à partir de node:20-alpine. Deux catégories de CVE ont été identifiées : des CVE sur les paquets OS Alpine (libssl3 et libcrypto3, liés à OpenSSL), et des CVE dans les dépendances JavaScript embarquées dans le CLI npm de l'image de base (tar, glob, minimatch, cross-spawn). Ces bibliothèques ne sont pas utilisées au runtime de l'application mais sont présentes dans l'image finale, augmentant inutilement la surface d'attaque. Le job Build & Scan Trivy s'est terminé avec exit code 1, bloquant l'exécution du job Deploy.

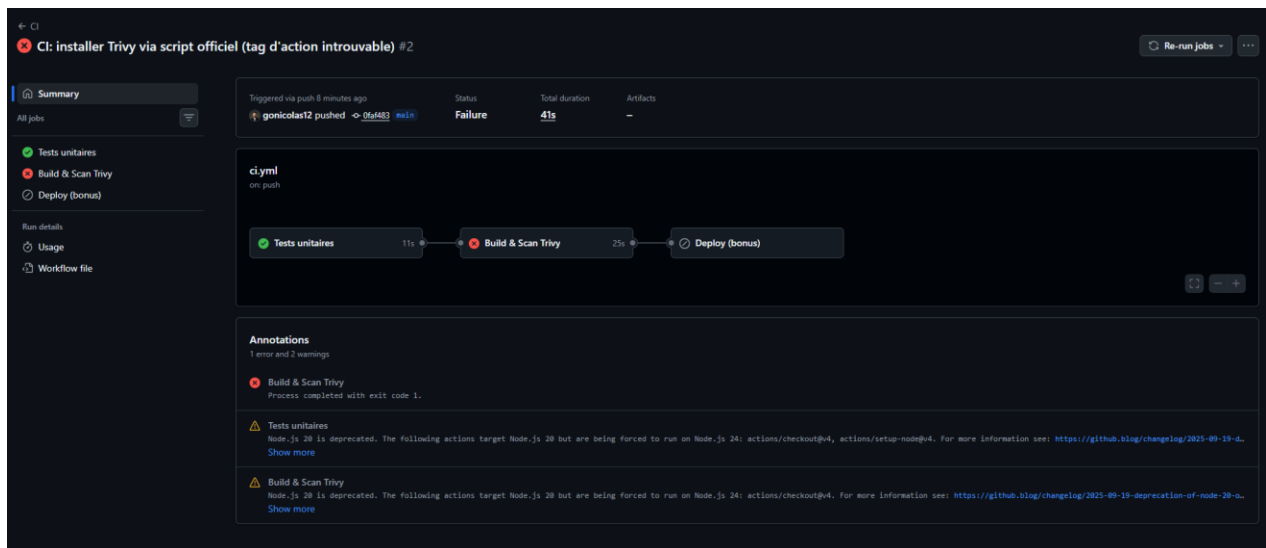


Figure 6 — GitHub Actions, run #2 : le job "Build & Scan Trivy" échoue (exit code 1) suite à la détection de CVE HIGH par Trivy ; le job "Deploy" ne s'exécute pas

B.2 — Pipeline vert : correction par durcissement du Dockerfile

La correction a été apportée directement dans le Dockerfile, sans contournement ni suppression des règles Trivy. Deux mesures ont été mises en œuvre :

- Ajout de la commande `apk upgrade --no-cache` dans l'étape de build de l'image Alpine, ce qui met à jour les paquets du système d'exploitation (dont libssl3 et libcrypto3) vers leurs dernières versions corrigées.
- Suppression du CLI npm de l'image finale : npm n'étant pas nécessaire au runtime d'une application Node.js en production, il a été retiré en supprimant son dossier : `rm -rf /usr/local/lib/node_modules/npm` + les binaires `npm/npx`, éliminant ainsi toutes les dépendances transitives vulnérables qu'il embarquait.

Ces deux corrections ont suffi à faire passer Trivy sans aucune CVE HIGH ou CRITICAL, permettant au pipeline de s'exécuter intégralement jusqu'au job Deploy.

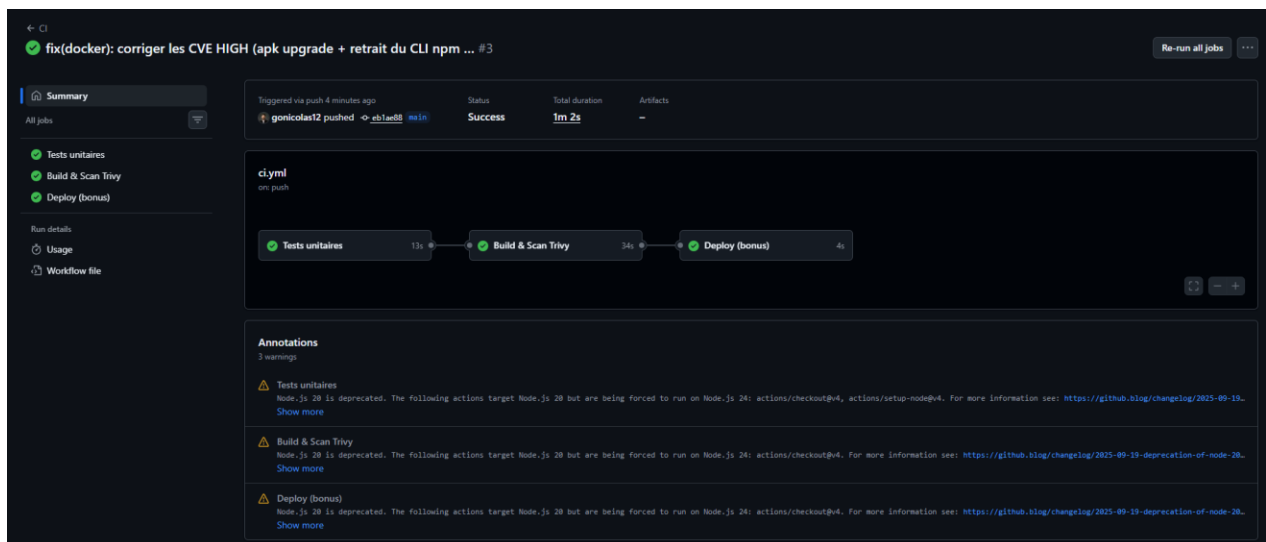


Figure 7 — GitHub Actions, run #3 "fix(docker): corriger les CVE HIGH (apk upgrade + retrait du CLI npm)" : les trois jobs Tests unitaires, Build & Scan Trivy et Deploy (bonus) passent au vert en 1 min 2 s

4. Choix techniques

Le parcours Docker Compose a été retenu en priorité car la stack TaskBoard était déjà entièrement définie dans un docker-compose.yml ; il suffisait d'y ajouter les services de supervision (Prometheus, node_exporter, cAdvisor, Grafana, Alertmanager, MailHog) en préservant l'isolation réseau existante. MailHog a été choisi comme faux serveur SMTP car il ne nécessite ni compte réel ni configuration TLS, et expose une interface web qui permet de vérifier visuellement la réception des alertes. Alertmanager a été configuré pour router une alerte unique (NodeExporterDown) vers deux receivers simultanément (mail et Discord), ce qui démontre la capacité de multi-notification sans dupliquer les règles d'alerte dans Prometheus. Le support natif de discord_configs dans Alertmanager v0.27 évite tout webhook personnalisé ou script intermédiaire. Côté CI/CD, le pipeline a délibérément séparé le job de tests du job de build+scan afin de fournir un feedback précis : un échec sur les tests ne lance pas de build inutile, et un échec Trivy ne masque pas un problème logiciel. Enfin, la correction des CVE a été traitée par durcissement du Dockerfile (mise à jour OS + suppression du CLI npm superflu) plutôt que par un fichier .trivyignore. L'option --ignore-unfixed reste active mais n'écarte que les CVE sans correctif disponible, sur lesquelles aucune action n'est possible.

5. Conclusion

L'ensemble des objectifs du TP a été atteint : la stack de supervision est opérationnelle avec Prometheus collectant les métriques de quatre cibles, Grafana visualisant l'état de l'infrastructure en temps réel, et l'alerte NodeExporterDown se propageant correctement vers MailHog et Discord via Alertmanager. Le pipeline CI/CD automatise tests, construction et analyse de sécurité à chaque push, avec une détection effective des CVE et leur résolution par durcissement du Dockerfile.

Les points bonus ont également été réalisés : l'étape Deploy est présente dans le pipeline et passe au vert après la correction des vulnérabilités, et le Dockerfile final intègre des mesures de réduction

de surface d'attaque (apk upgrade + suppression du CLI npm) conformes aux bonnes pratiques de sécurité des images de conteneurs.